Google Antigravity Architecture

Complete Technical Guide: Agentic Platform, Customization Roots & Rule Enforcement

Comprehensive Reference Manual: *This unified document compiles all core architectural pillars of Google Antigravity: (1) The paradigm shift from passive IDE editor to autonomous agentic runtime, (2) Global vs. Workspace customization hierarchies (~/.gemini/config vs. .agents/), and (3) The complementary mechanisms of ambient rules (AGENTS.md) vs. on-demand progressive disclosure skills (SKILL.md).*

Module 1: Antigravity as an Agentic Platform vs. IDE Editor

Traditional IDEs place the human at the center of keystrokes, with AI operating as a passive autocomplete assistant. Antigravity inverts this: the AI agent acts as an autonomous operator driven by goals, executing shell commands, managing file trees, conducting multi-step planning, and verifying outcomes against test suites.

Dimension	Traditional IDE Editor	Antigravity Agentic Platform
Primary Operator	Human developer typing code line-by-line.	Autonomous agent executing multi-step goal loops.
Execution Scope	Current cursor context or active file buffer.	Full OS environment: shell, git, browser, APIs, filesystem.
Verification	Manual: Developer copies code, runs tests, reads errors.	Automated: Agent executes test suites, analyzes logs, and self-repairs.
Concurrency	Single-threaded human focus.	Parallel subagents, background tasks, cron daemons.

Module 2: Global vs. Workspace Customization Roots

Antigravity manages configuration across two distinct root directories to balance personal developer ergonomics with team reproducibility:

Attribute	Global Root (~/.gemini/config/)	Workspace Root (.agents/)
Scope & Audience	Machine-wide across all user projects; private to local machine.	Project-scoped; checked into Git and shared with team.
Loading Precedence	Lower priority (serves as personal default / fallback).	Highest priority (overrides global and built-in configurations).
Typical Contents	Personal CLI utilities, private credentials, developer dotfiles.	Team coding standards, CI/CD runbooks, repo-specific MCP configs.

Module 3: AGENTS.md Operational Rules vs. SKILL.md Frontmatter

To prevent prompt clutter while maintaining strict behavioral standards, Antigravity splits customization into two mechanics:

- **AGENTS.md / GEMINI.md (The Guardrail):** Ambient, hierarchical Markdown rules that apply automatically to directory subtrees. Ideal for architectural invariants, forbidden libraries, and test policies.
- **SKILL.md (The Playbook):** Modular runbooks triggered on-demand via YAML frontmatter description matching. Leverages Progressive Disclosure to inject rich procedures only when relevant.

Feature	Operational Rules (AGENTS.md)	Triggered Skills (SKILL.md)
Activation	Ambient & always-on within directory tree.	Event-driven on-demand via prompt intent matching.
Frontmatter	None (pure Markdown).	Mandatory YAML frontmatter (name, description).
Context Strategy	Directly loaded when operating in folder scope.	Progressive disclosure (zero overhead until triggered).

Standard Workspace & Customization Layout

```
/
■■■ .agents/
■   ■■■ skills/
■   ■   ■■■ /
■   ■       ■■■ SKILL.md       # Mandatory YAML frontmatter + runbook
■   ■       ■■■ scripts/       # Executable helper utilities
■   ■       ■■■ references/    # Bulky documentation loaded on demand
■   ■■■ rules/                 # Additional contextual rule markdown files
■   ■■■ hooks.json             # Lifecycle event triggers
■   ■■■ mcp_config.json        # Workspace-scoped MCP tool configurations
■■■ AGENTS.md                 # Root-level ambient operational rules
■■■ src/ ...
```

Project Policy Notice: As instructed, all explanations, answers, and technical reports for this project will henceforth be generated directly in professional PDF format and saved into the project workspace.